



Все курсы



Тота ИИ



Шаг пройден

[🏠](#) / [Мое](#) / [Мои проекты](#) / [AI-агент службы поддержки](#)

Проект: AI-агент службы поддержки

15

августа
2026 г.

> НЕД...

📅 НЕД...

◦ В П...

[Репозиторий](#)[Помощь](#)

Подготовка

✓ 1

✓ 2

✓ 3

✓ 4

✓ 5

✓ 6

> 7

> 8

> 9

> 10



Проверка

Подключение RAG: Help Desk

Загружаем корпоративную базу знаний (документация, FAQ, регламенты) в векторное хранилище Яндекса и подключаем её к агенту через инструмент `file_search`. После этого шага бот превращается в полноценный Help Desk: отвечает на вопросы пользователя на основе документов, а не придумывает ответ.

К этому моменту агент уже умеет:

- общаться с пользователем по почте (шаг про `email-workflow`),
- сохранять обращения в YDB через MCP `ydb-tickets` (шаги про MCP и авто-эскалацию).

Но без базы знаний он галлюцинирует. RAG закрывает этот пробел: перед ответом агент ищет релевантные документы и формирует ответ



Все курсы

...

```

1  пользователь: "как оформить командировку?"
2  |
3  ▼
4  агент → file_search → top-5 релевантных документов
5  |
6  ▼
7  агент формирует ответ на основе документов + ссылается на них
8  |
9  ▼
10 если ответа нет → через MCP ydb-tickets создаётся тикет в YDB

```



Ссылки

- [Поисковые индексы Vector Store](#) — документация Yandex AI Studio

Задачи

1. Подготовьте корпус документов (markdown / PDF). Минимум — 5-10 документов про HR / IT / администрирование. Если документы разбиты по темам — решите: либо один большой индекс, либо несколько меньших (см. ниже ограничение file_search).
2. Создайте search index — через CLI `yandex-ai-studio vector-stores local` (локальные файлы), `s3` (бакет), `confluence`, `wiki`. Полный список флагов — `yandex-ai-studio vector-stores local --help`. Доступны `--name`, `--max-file-size`, `--expires-after-days`, `--poll-timeout`. Выбор embedding-модели происходит на стороне AI Studio (подробнее — в блоке «Ссылки»).

```

1  source .venv/bin/activate
2  yandex-ai-studio vector-stores local docs/*.md docs/*.pdf --name "

```





Все курсы

3. Получите ID уже созданных индексов (если они создавались в UI AI Studio или другим путём) — через `yandex_ai_studio_sdk`:

```
1 from yandex_ai_studio_sdk import AIStudio
2 ai = AIStudio() # подхватит YC_FOLDER_ID и YC_IAM_TOKEN из env
3 for idx in ai.search_indexes.list():
4     print(idx.id, idx.name)
```

ис CLI для `search indexes` НЕТ — только SDK или UI.

4. Подключите индекс к агенту через `file_search` tool в Responses API. Никакого UI-шага не нужно, если вы вызываете агента inline (как в функции `email-poller`):

```
1 "tools": [
2     {
3         "type": "file_search",
4         "vector_store_ids": [<single_search_index_id>],
5     },
6     # ... другие tools (MCP, function)
7 ]
```

5. Пропишите в инструкциях (поле `instructions` в Responses API, не UI) поведение агента: перед ответом обращаться к базе знаний через `file_search`; при найденном ответе — коротко + ссылка на документ; при отсутствии — честно сказать «не знаю» и предложить создать тикет через `create-ticket`. Точную формулировку промпта подберите сами.

6. Проверка: вопрос из базы → в `output[]` видно `type:`

`"file_search_call"` с результатами → ответ с цитатой и ссылкой.

Вопрос вне базы → агент честно говорит «не знаю» и предлагает тикет.



явно «перед ответом ВСЕГДА обращай к базе знаний через `file_search` ».

- Если агент цитирует документ целиком — добавьте в промпт «не цитируй больше 3 предложений, дай краткое резюме и ссылку».
- Если топ-1 документ нерелевантен (низкий `score`) — лучше ответить «не знаю»; поднимите порог релевантности в настройках `tool'a`.

Результат шага

- Вы задаёте боту вопрос из базы знаний — получаете ответ со ссылкой на документ (видно `file_search_call` в трейсе).
- Вы задаёте вопрос вне базы — бот честно говорит «не знаю» и предлагает создать тикет; в YDB появляется новая запись (`use serverless function invoke ydb-tickets --data '{"action":"list-my-tickets",...}'`).
- Вы можете показать в трейсе: какой документ был найден, какие `queries` агент сгенерировал.

Ограничения Yandex Responses API для `file_search` (важно!)

- Только один `vector_store_id` за раз. Передача массива из нескольких IDs даёт `400 Bad Request: "FileSearch support only one vector store id"` .
- Только один `file_search` `tool` за раз. Попытка добавить два инструмента `file_search` в один запрос даёт `400 Bad Request: "Filesearch tool was given more than once"` .
- Следствие: если у вас 3 независимых корпуса (например, `start` , `code-errors` , `payments`), их надо объединить в один `search index` при создании (`yandex-ai-studio vector-stores local start/*.md errors/*.md`)